Class #5

03.01 Enterprise Software & Domain-Driven Design

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Agenda

- **Block 1 — Enterprise Systems as a Design Problem** 55 min
  - Enterprise systems as trade-off space
  - L0 / L1 / L2 architecture levels
- **Block 2 — Strategic Design, Boundaries, and Consistency** 100 min
  - Why Domain-Driven Design is important
  - Strategic design and subdomain classification
  - Exercise 03.01.01
  - Bounded contexts and context relationships
  - Tactical DDD essentials
  - Aggregate boundaries under load
  - Exercise 03.01.02
  - Domain events and cross-aggregate rules
  - From discovery to executable language

O'Reilly Playlist · learning.oreilly.com/playlists/e4bbd2ea-5604-419c-8f40-2b0a06e7f8aa

⏱ 0:00 – 55 min

Block 1

Enterprise Systems as a Design Problem

UrbanEats, 2019 vs 2023

2019

3 developers. One database. One API. One country. Deployments on Fridays.

It worked.

2023

40 million orders/month. 12 teams. 5 countries. 4 payment providers. Real-time inventory. ML recommendations. The original codebase is still in there somewhere.

> What changed? Load, teams, integrations, and operational expectations. The domain stayed recognizable. The design problem did not.

URBANEATS · 2019

If you were the CTO in 2019, what would you have designed differently, knowing where the company would be in 2023?

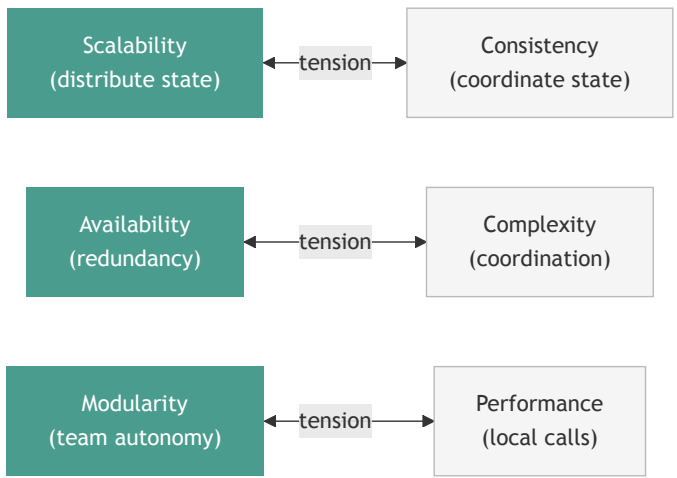
Enterprise systems become hard in three ways

Pressure	What grows	Architectural consequence
Scale	More users, more requests, more concurrency	State, contention, and failure handling become design problems
Coordination	More teams, more handoffs, more releases	Ownership and boundaries matter as much as code quality
Integration	More vendors, channels, and regulators	Translation and contract stability become first-class concerns

Enterprise systems are not defined by company size. They are defined by the cost of getting the boundaries wrong.

Architecture starts when trade-offs stop being optional

These tensions do not go away. They only become more expensive.



Architecture is the discipline of making these trade-offs deliberately.

Three levels, three different decisions

These three levels answer different architectural questions.

LEVEL	CORE QUESTION	PRIMARY USE
Level 0 Business Architecture	What capabilities does the business need?	Business leaders, product, domain experts Capability maps · value streams · stakeholders
Level 1 Technical Architecture	What systems and boundaries deliver those capabilities?	Architects, tech leads, delivery teams Contexts · services · APIs · data flows
Level 2 Deployment Architecture	How does the solution run, scale, and recover?	Platform, SRE, operations Topology · scaling strategy · resilience mechanisms

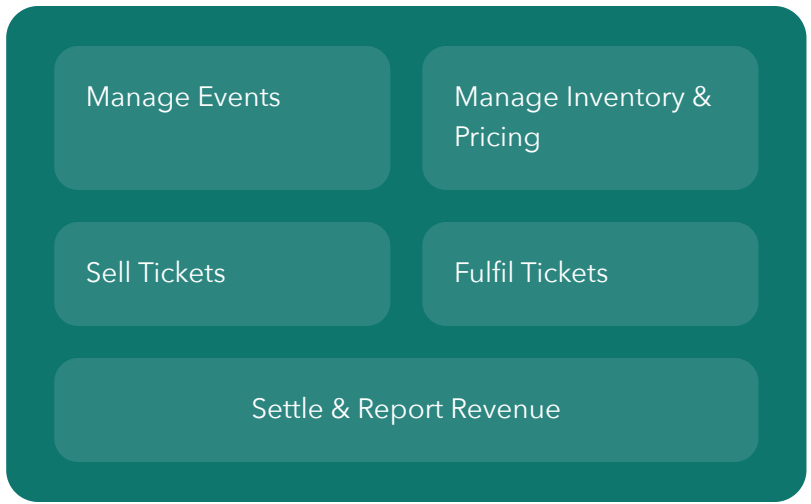
L0 — Business Architecture

What does the organization need to do, and who cares about it?

STAKEHOLDERS

- Customer
- Promoter
- Box Office
- Finance

CORE CAPABILITIES



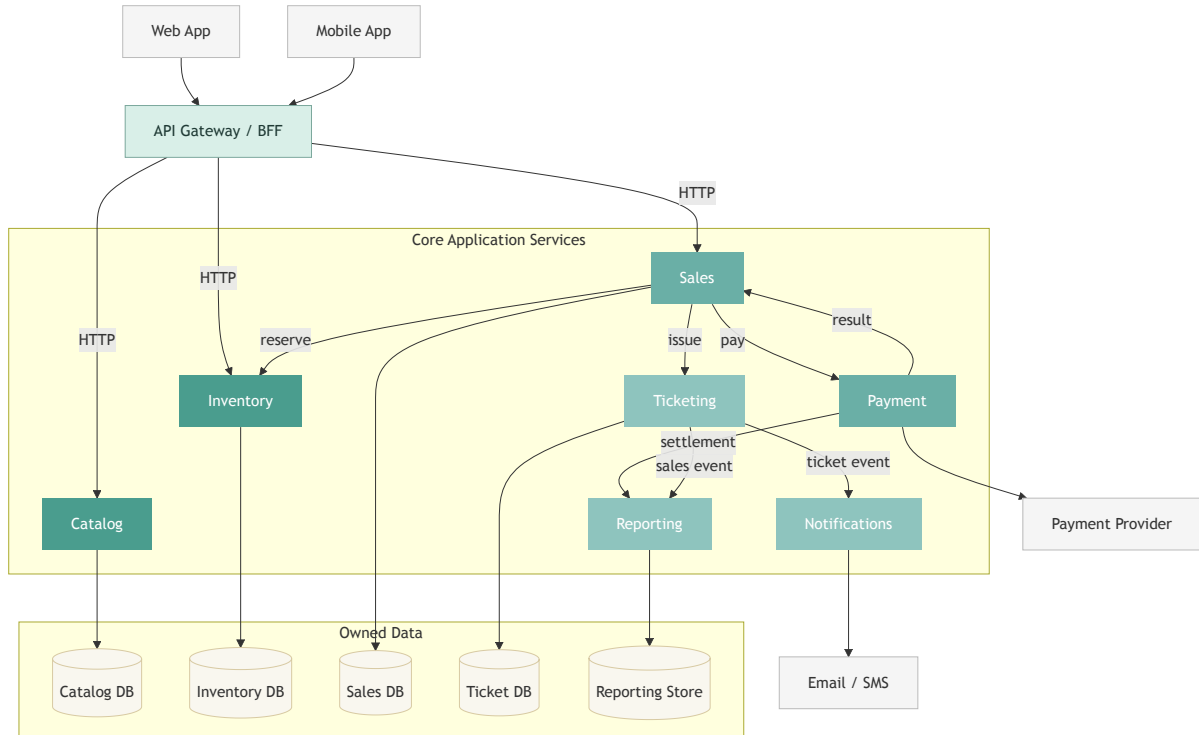
BUSINESS OUTCOMES

- Events reach the market on time
- Seat inventory and pricing stay under control
- Customers can discover, buy, and receive tickets reliably
- Revenue becomes visible, auditable, and reconcilable

No services. No databases. No technology. Only capabilities, actors, and business outcomes.

L1 – Technical Architecture

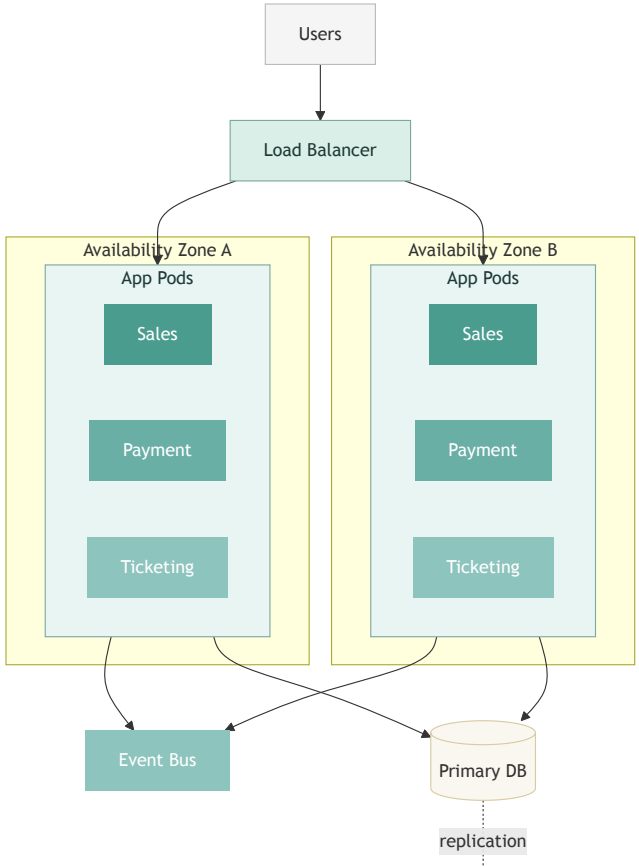
What systems and boundaries will deliver those capabilities?



This is where business capabilities become software systems, APIs, owned data, and integration contracts.

L2 – Deployment Architecture

How does the solution run under load, fail, recover, and scale?



📖 Revisit — Home Reading

Enterprise Software Systems

The live lecture framed enterprise systems as a trade-off problem and used L0/L1/L2 to position DDD. This section preserves the fuller reference material: the complete characteristic set, measures, and richer examples.

The 10 characteristics — how to use this reference

The live lecture focused on enterprise systems as a trade-off space. The full set of 10 below is the standard vocabulary for evaluating enterprise systems.

Use this table when:

- Reviewing a system's design for a quality attribute you suspect is under-served
- Writing quality attribute scenarios (the 6-part structure: source, stimulus, artefact, environment, response, measure)
- Arguing for or against an architectural decision — the "typical measure" column gives you the metric to attach to a requirement

Reusability and **Replaceability** were not treated live. They are relevant primarily to platform and shared-library decisions and become important again in Chapter 4 (Architecture Methodologies).

The 10 characteristics — full reference

Property	Definition	Typical measure
Scalability	Capacity grows with load, horizontally or vertically	Requests/sec at target latency under 2× load
Availability	System remains operational despite partial failures	% uptime (99.9% = 8.7h downtime/year)
Latency	Time from request to response under real conditions	p95/p99 response time in ms
Robustness	Graceful degradation on unexpected inputs or failures	% requests handled without data loss on failure
Security	Protection of data and operations from unauthorised access	CIA model compliance: Confidentiality, Integrity, Availability
Modularity	Independent development, deployment, and failure isolation	Team deployment frequency without cross-team coordination
Reusability	Components serve multiple consumers without modification	Number of consumers per shared component
Replaceability	Components can be swapped without cascading changes	Time to replace a component without touching consumers
Observability	System state can be inferred from external outputs	Mean time to detect an incident (MTTD)
Adaptability	New requirements absorbed without architectural rewrites	Cost of change over system lifetime

The full trade-off map

Maximising this…	…creates tension with
Scalability (distribute state)	Consistency (centralising state enforces one truth)
Availability (redundancy everywhere)	Complexity (coordinating replicas is hard)
Latency (local caches, in-process calls)	Consistency (cached data goes stale)
Modularity (separate services)	Performance (network calls add latency)
Security (encryption, auth checks)	Latency (every request pays the crypto overhead)
Reusability (shared libraries)	Replaceability (consumers couple to the shared version)
Observability (emit everything)	Performance (logging and tracing have overhead)
Adaptability (loose coupling)	Simplicity (abstractions add indirection)

There is no architecture that maximises all ten. The architect's job is to make trade-offs **deliberately and explicitly**, and to document why.

Availability — the numbers

$$\text{Availability} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

Where MTBF = Mean Time Between Failures, MTTR = Mean Time To Repair.

SLA target	Downtime per year	Downtime per month
99%	87.6 hours	7.3 hours
99.9% ("three nines")	8.76 hours	43.8 minutes
99.99% ("four nines")	52.6 minutes	4.4 minutes
99.999% ("five nines")	5.26 minutes	26.3 seconds

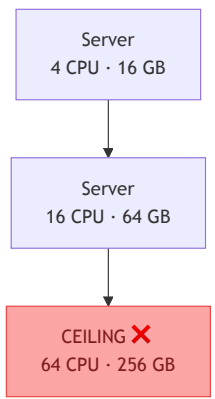
Each additional nine roughly multiplies architecture complexity by 10×.

Scalability — the two axes

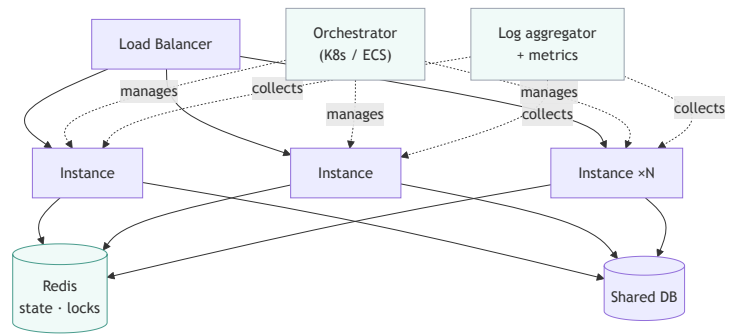
Vertical scaling is simple: pay more, get a bigger machine, the system works exactly as before — until it hits the hardware ceiling.

Horizontal scaling is not "add more instances." It shifts complexity from the infrastructure budget into the engineering team. Before you can scale out, you have to rearchitect.

⬆️ Vertical — pay more, hit ceiling



➡️ Horizontal — unbounded, but you pay in complexity



Horizontal scaling — what changes

Scaling out is not primarily an infrastructure move. It is a state-management and coordination move.

What horizontal scaling actually requires before the first new instance is added:

Requirement	Why
Stateless services	Instance A cannot hold session data that Instance B doesn't see
Externalized state	Sessions, distributed locks, and caches move to Redis or the DB
Orchestration	Something must start, stop, and health-check N instances
Aggregated observability	Logs and metrics from N instances must be collected centrally
Failure handling	Network partitions and split-brain are new failure modes with no vertical equivalent

RocketTickets flash sale: a seat held on Instance A must be visible to Instance B instantly. Without Redis as the shared lock store, the same seat gets sold twice.

Solution architecture levels — overview

The three levels answer different architectural questions. They are often produced in sequence, but revised iteratively: a runtime constraint at L2 may force a redesign at L1 or even a rethink at L0.

Level	Core question	Primary users	Typical outputs
L0 — Business	What capabilities does the business need?	Business leaders, product, domain experts	Capability maps, value streams, stakeholder maps
L1 — Technical	What systems and boundaries deliver those capabilities?	Architects, tech leads, delivery teams	Context/service decomposition, APIs, data flows, integration contracts
L2 — Deployment	How does the solution run, scale, and recover?	Platform, SRE, operations	Runtime topology, scaling strategy, resilience mechanisms, failover plans

L0 — Business Architecture example

RocketTickets: what does the business need to do, and who is involved?

Stakeholders

- Customer
- Promoter
- Box Office
- Finance

Core capabilities

- Manage events
- Manage inventory & pricing
- Sell tickets
- Fulfil tickets
- Settle & report revenue

Business outcomes

- Events reach the market on time
- Seat inventory and pricing stay under control
- Customers can buy and receive tickets reliably
- Revenue becomes visible and reconcilable

This is the standard L0 view: capabilities, stakeholders, and business outcomes. No services. No databases. No technology decisions.

L0 – Ownership and policy detail

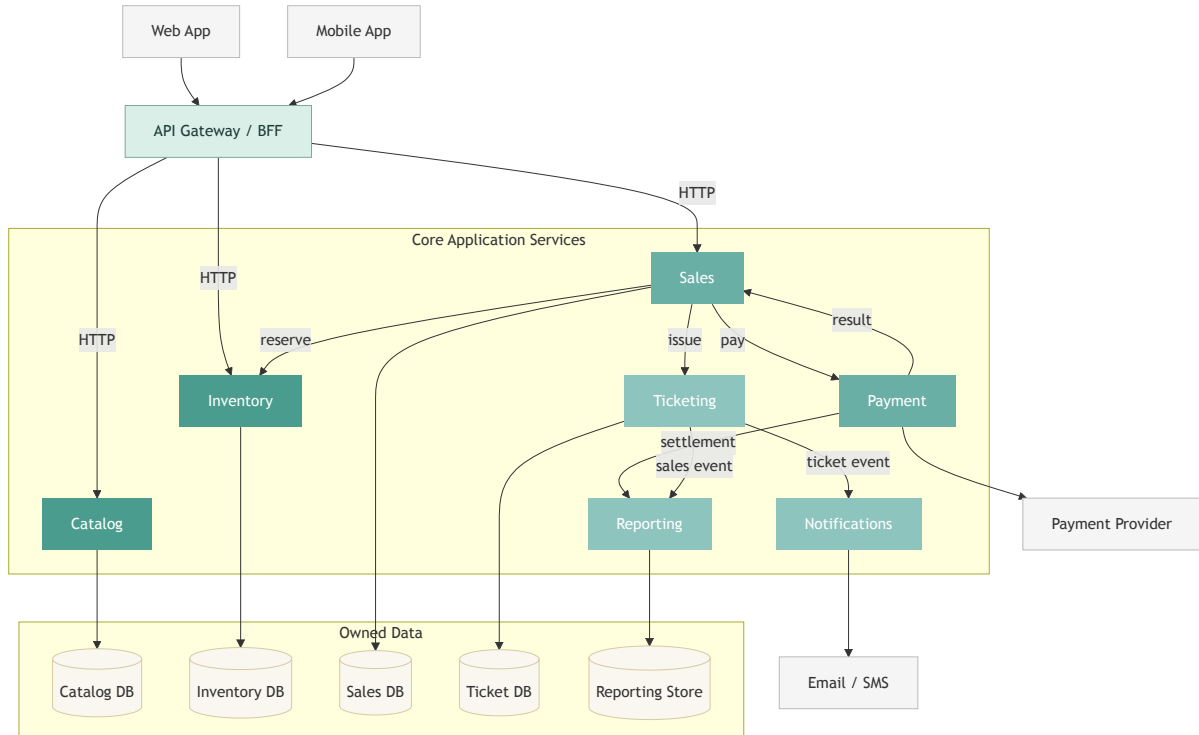
The same business view can be made more concrete by attaching ownership and policy to each capability.

Capability	Primary owner	Concrete policy / example
Manage events	Promoter operations	Publish an event only after venue, schedule, and seating plan are approved
Manage inventory & pricing	Box Office + Promoter	Hold back VIP seats, open extra sections, change prices when demand spikes
Sell tickets	Commercial / digital sales	Apply promo codes, enforce purchase limits, expose availability to web/mobile
Fulfil tickets	Customer operations	Deliver QR code tickets, resend on request, support venue scanning
Settle & report revenue	Finance	Reconcile payment batches, calculate promoter payout, track refunds and chargebacks

This is still L0. The content is richer, but it remains business-facing: ownership, operating policy, and business rules rather than software structure.

L1 – Technical Architecture example

RocketTickets: what systems deliver the Ticket Sales capability?



At this level, business capabilities become software systems, APIs, owned data, and integration contracts.

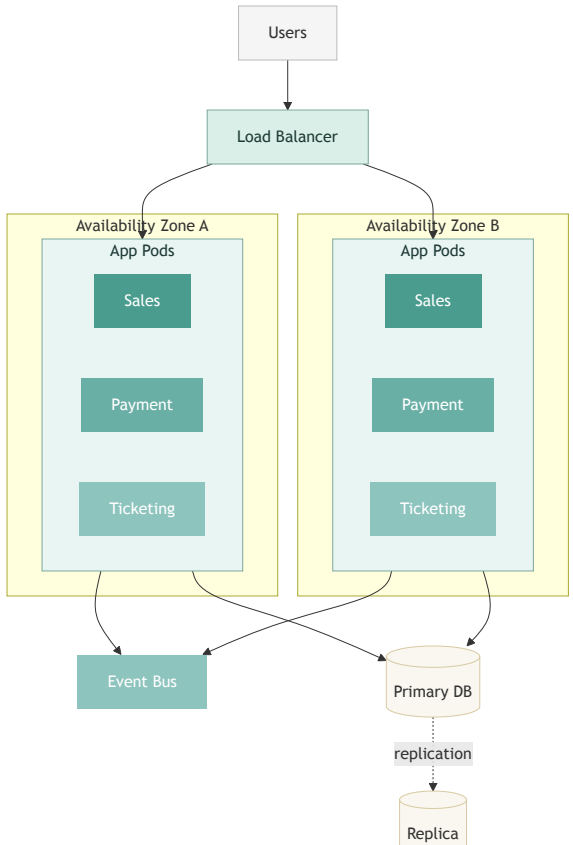
L1 — Interfaces and ownership detail

The same technical view can be made more concrete by attaching data ownership and interface shape to each system.

System	Owns	Key interface	Integration style
`Catalog`	event metadata	`GET /events` , `GET /events/{id}`	synchronous read API
`Inventory`	seat availability and holds	`POST /holds` , `DELETE /holds/{id}`	synchronous command API
`Sales`	carts and reservations	checkout orchestration	coordinates multiple downstream calls
`Payment`	payment state	provider adapter + webhook handler	external provider integration
`Ticketing`	issued tickets	consumes confirmed-sale event	asynchronous event consumer
`Reporting`	sales facts	reporting feed	append-only async ingestion

This is the standard L1 extension: responsibilities, boundaries, interfaces, data ownership, and integration style.

L2 – Deployment Architecture example



RocketTickets flash sale: how does it run under 50 000 concurrent users?

What this shows:

- multiple app services deployed in each zone
- traffic distributed across zones
- shared runtime dependencies
- replicated state for recovery

Concern	Runtime tactic
Load spikes	scale app pods horizontally behind the load balancer
DB pressure	keep one primary for writes and one replica for reads/reporting
Zone failure	run the same app set in two availability zones
Coordination	use the event bus instead of direct synchronous chains everywhere

Audience: platform engineers, site reliability engineers, and operations

Where DDD fits across the levels

Domain-Driven Design contributes to L0 and L1. L2 is infrastructure and is largely independent of domain design.

Level	DDD contribution
L0	*Strategic design* — subdomain classification (Core / Generic / Supporting), bounded context identification, context map. These decisions determine which capabilities get engineering investment and which are bought.
L1	*Tactical design* — entities, value objects, aggregates, repositories, domain events. These patterns structure the inside of each bounded context and determine whether the code can evolve as the domain grows.
L2	None directly. L2 decisions (cloud provider, container orchestration, replication) are driven by the quality attributes (availability, scalability) defined at L1, not by domain design.

Further reading

- **Learning Domain-Driven Design** – Vlad Khononov (O'Reilly) – best modern introduction, covers strategic and tactical design
- **Domain-Driven Design: Tackling Complexity in the Heart of Software** – Eric Evans (Addison-Wesley) – the original text; dense but authoritative
- **Software Architect's Handbook** – Joseph Ingeno (O'Reilly) – broader context: enterprise characteristics, architectural roles
- **Fundamentals of Software Architecture** – Richards & Ford (O'Reilly) – trade-offs, architecture patterns, decision-making

⏱ 1:00 – 100 min

Block 2

Strategic Design, Boundaries, and Consistency

Why Is Domain-Driven Design Important?

We often want to discuss architectural features and quality attributes early:

- scalability
- modifiability
- availability
- security

But those discussions stay vague until the system is broken into manageable parts.

You cannot reason about those qualities well if you have not first decided:

- what the important subdomains are
- where one context ends and another begins
- which consistency boundaries are expensive under load

Before evaluating architectural qualities, you need candidate boundaries. Domain-Driven Design gives you those first manageable units.

Where Domain-Driven Design Fits

Level	Domain-Driven Design contribution	Why it matters
L0	Strategic design: subdomains, priorities, business language	Decides where custom engineering effort belongs
L1	Bounded contexts, aggregates, repositories, domain events	Decides what belongs together and what must stay apart
L2	Pressure test	Concurrency, deployment, and failure modes expose bad boundaries

Domain-Driven Design is not an alternative to architecture. It is how architecture becomes meaningful inside the problem space.

Strategic design starts with one question

> What do we build ourselves, and what do we refuse to spend our best engineers on?

Your team is building a last-mile delivery platform.

Capability	Decision pressure
Route optimization	Product advantage or solved problem?
Surge pricing	Strategic differentiator or commodity SaaS?
Driver onboarding	Compliance cost or competitive edge?
Notifications	Domain logic or channel plumbing?
Accounting & invoicing	Product capability or off-the-shelf integration?

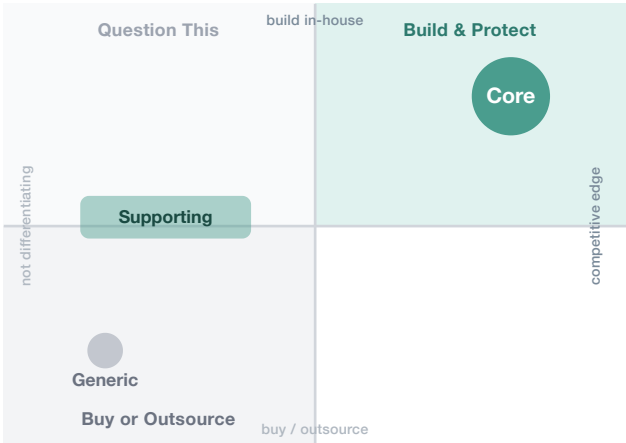
The answer is not technical first. It is strategic first.

Strategic design produces three artefacts

Artefact	What it tells you
Subdomain map	Where the business wins, supports, or commoditizes
Bounded contexts	Where one model must stop and another must begin
Context relationships	How those contexts integrate without polluting each other

This is why DDD belongs in an architecture course. It turns business ambiguity into design boundaries.

Domain, subdomain, and investment



Type	Strategy	Investment
Core	Build and protect in-house	High
Supporting	Build simply	Low
Generic	Buy, outsource, or use open source	Near zero

The classification is about competitive importance, not technical difficulty.

The same capability, different companies

Capability	Company A	Company B
Fraud detection	**Generic** – use a third-party fraud API	**Core** – fraud accuracy is the product
Route optimization	**Generic** – Google Maps is fine	**Core** – faster delivery is the edge
Medical image analysis	**Generic** – use a platform service	**Core** – model accuracy differentiates them
Search ranking	**Generic** – defaults are enough	**Core** – relevance is the business

> Subdomain type is not a property of the feature. It is a property of the feature's relationship to strategy.

Exercise 03.01.01

Strategic Domain Classification

A company description with competing strategic visions. Classify its subdomains and defend every choice.

Exercise document →

docs.google.com/document/d/1Xa1IZOMrprsIualsKvhnEGLLhbwrap3kvlSWV0sjeCo

30'
DURATION

3–4
PER GROUP

Debrief — what to keep from Exercise 03.01.01

- **Subdomain type is relative to strategy**: the same capability can be Core in one company and Generic in another
- **A good boundary aligns owner, language, and reason to change**
- **Your output is not a taxonomy**: it is a first map of where architectural attention should go

Carry forward:

- candidate Core / Supporting / Generic subdomains
- contested boundaries worth pressure-testing
- first clues about where consistency and coupling will be expensive

When should you draw a boundary?

Once you have candidate subdomains, the next question is where one model should stop and another should begin.

Draw a boundary between two things when:

- a different team owns one side, or should
- the same word means something different on each side
- they change for different business reasons
- one side could be replaced or rewritten without dragging the other with it

Stop when each candidate has:

- one clear owner
- one internal language
- one primary reason to change

Two ways to find the boundary

Top-down

Start from the business. Identify capabilities. Classify them. Drill into the Core and Supporting areas until the stop rule fires.

Failure mode: staying too abstract and missing the real fault lines inside one capability.

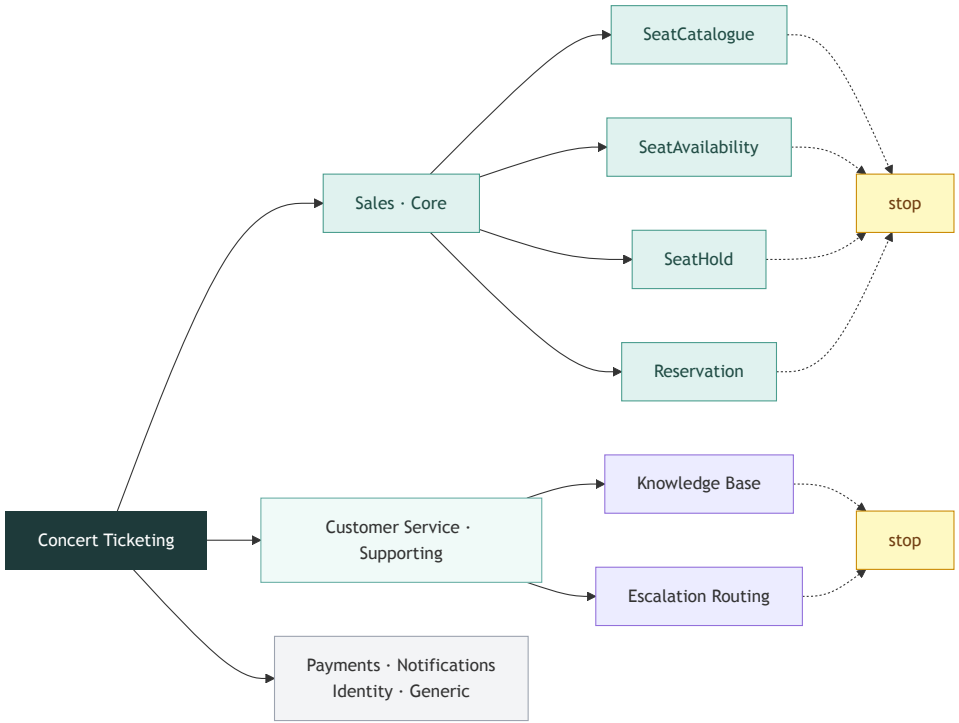
Bottom-up

Start from existing teams, code, incidents, and language. Group what clearly belongs together. Merge until ownership or meaning would be lost.

Failure mode: over-decomposing too early and inventing boundaries that no one can own.

RocketTickets from above

Identify, classify, drill, stop, redraw if needed.



RocketTickets from below

Start from what the organization already names, owns, and changes.

SeatCatalogue
SeatAvailability
SeatHold
Reservation
PaymentGateway
PaymentRetry
TicketIssuance
TicketDelivery
EmailNotification
CustomerProfile
CustomerSession

Group	Type	Why it survived
Sales	Core	One owner, one seat-state vocabulary, one reason to change
Payments	Generic	Stable workflow and failure modes; buy Stripe
Ticketing	Supporting	One issuance lifecycle, one owner
Notifications	Generic	Channel concern, not business domain
Identity	Generic	Mature external solutions, no differentiation

Merging any two groups would blur ownership or language. Stop.

Bounded Contexts

When one word carries different meanings for different teams, one model is already failing.

Billing team

An `Order` is a financial record with invoice reference, tax breakdown, payment state, and audit trail.

Warehouse team

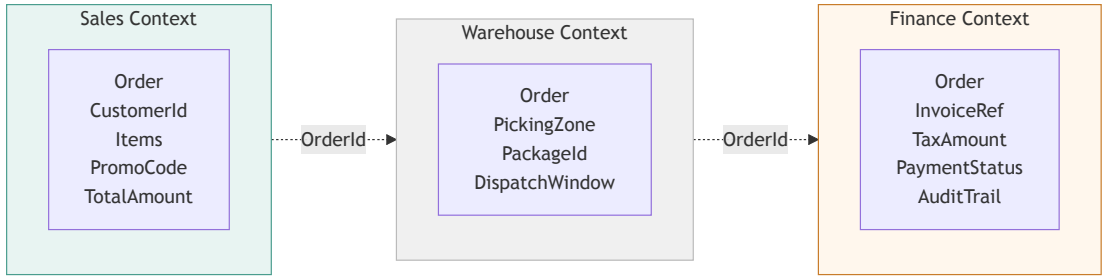
An `Order` is a picking job with zone, package identifier, and dispatch window.

Customer support team

An `Order` is the thing the customer placed and wants to track.

One class serving all three teams is the architectural version of low cohesion.

One word, three models



Each context owns its own model. They may share an identifier. They do not share a class.

Context relationships matter too

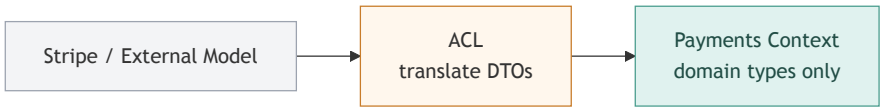
Boundaries alone are not enough. Contexts also need disciplined translation rules.

Pattern	Use it when
Customer-Supplier	One team depends on another team's model and contract
Conformist	Downstream accepts upstream language as-is because it has no leverage
Anti-Corruption Layer	Downstream must protect its own model from upstream pollution
Open Host Service	One context exposes a stable protocol to many consumers

The most important default for enterprise systems: **use an ACL unless you have a good reason not to**.

Anti-Corruption Layer

The ACL is the translation membrane at the boundary.



- External DTOs stop here
- Translation happens here
- Domain concepts continue inward

Without an ACL, external model decisions leak directly into your core language.

Anti-Corruption Layer in C#

The boundary should be visible in code too.

```
// external DTO from Stripe webhook
public record StripePaymentResult(
    string reservation_id,
    long amount_received_cents,
    string currency,
    long confirmed_at_unix);

// domain type inside the Payments context
public record PaymentConfirmation(
    Guid ReservationId,
    Money Amount,
    DateTimeOffset ConfirmedAt);

public class PaymentAcl {
    public PaymentConfirmation Translate(StripePaymentResult dto) {
        return new PaymentConfirmation(
            ReservationId: Guid.Parse(dto.reservation_id),
            Amount: new Money(dto.amount_received_cents / 100m, dto.currency.ToUpper()),
            ConfirmedAt: DateTimeOffset.FromUnixTimeSeconds(dto.confirmed_at_unix));
    }
}
```

- `StripePaymentResult` stops at the boundary
- `PaymentConfirmation` is what the domain understands
- if Stripe changes, the translation layer changes, not the domain model

From strategic boundaries to tactical design

Inside each bounded context, you still need a model.

Building block	Fast decision rule
Entity	Use it when identity must survive change
Value Object	Use it when equality by value is enough
Aggregate	Use it when several objects must remain consistent together
Repository	Use it when application code must load and save aggregates without speaking SQL

The tactical details matter. But at architecture level, the hardest question is usually the aggregate boundary.

Aggregate = consistency boundary

An aggregate is not "a bunch of related objects."

It is the set of things that must remain consistent **together**, in one transactional decision.

Usually that means:

- one **aggregate root** controls changes
- invariants inside the boundary must always hold when a transaction finishes
- anything outside the boundary can be updated later, through another transaction or an event

Ask one question first:

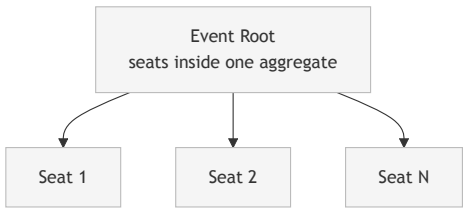
> What absolutely must not be observed in an invalid intermediate state?

If the answer is "not much," the aggregate should probably be small.

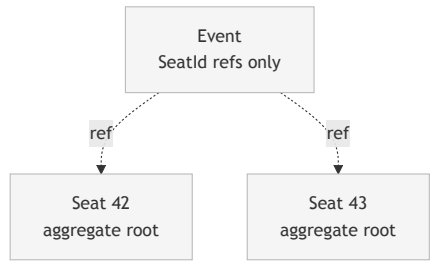
The mistake is to group objects because they are related in the data model. The real criterion is whether they must succeed or fail **together**.

How would you model the flash sale?

Option A: Event-centric aggregate



Option B: Seat-centric aggregates



Same domain. Two plausible cuts. Now ask which consistency boundary survives real load.

Exercise 03.01.02

Flash Sale Domain Model

Apply the aggregate-boundary question to the RocketTickets flash sale. Choose the boundary you would keep under real load, and justify it.

Exercise document →

docs.google.com/document/d/1JHBgK_mAuj_m8ljfnPX7yp0hCm0N8VZFd0l0DEGzM-s

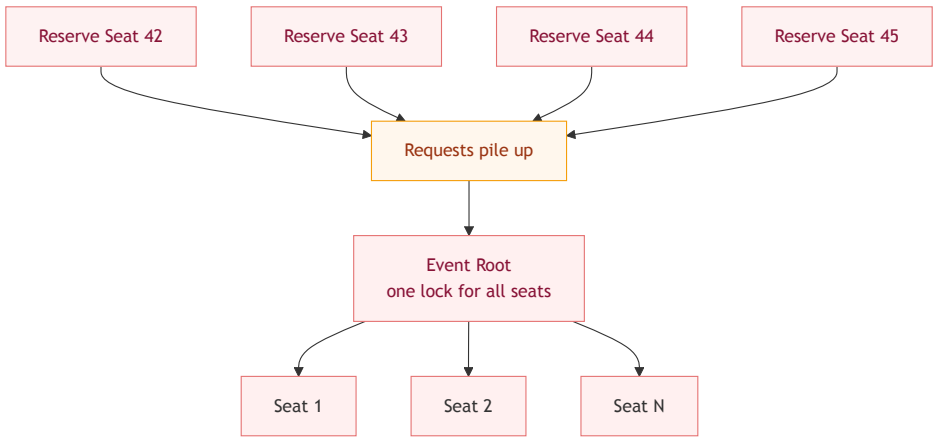
45'
DURATION

3–4
PER GROUP

The aggregate boundary trap

How would you model a RocketTickets flash sale?

Naive: `Event` **holds every** `Seat`



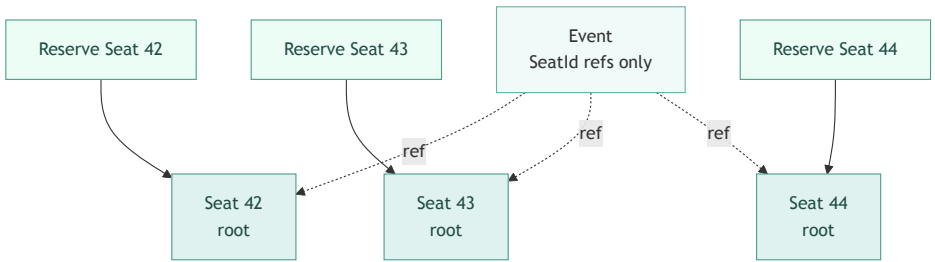

```
public class Event {
    private List<Seat> _seats;

    public async Task ReserveAsync(Guid seatId, Guid customerId) {
        _seats.Find(seatId).Reserve(customerId);
        await Task.CompletedTask;
    }
}
```

The entire event becomes one consistency boundary.

The aggregate boundary trap

How would you model a RocketTickets flash sale?

Correct: each `Seat` is its own root




```
public interface ISeatRepository {
    Task<Seat> FindForReservationAsync(Guid seatId);
    Task SaveAsync(Seat seat);
}

public class ReservationService {
    private readonly ISeatRepository _seats;

    public async Task ReserveAsync(Guid seatId, Guid customerId) {
        var seat = await _seats.FindForReservationAsync(seatId);
        seat.Reserve(customerId);
        await _seats.SaveAsync(seat);
    }
}

public class Seat {
    public void Reserve(Guid customerId) {
        if (Status ≠ Available)
            throw new SeatUnavailableException();
        Status = Held;
    }
}
```

Each seat becomes its own consistency boundary.

What changed when the aggregate changed

Design	Consistency scope	Performance consequence
`Event` **as root**	One giant lock for all seat changes	Serializes reservations under load
`Seat` **as root**	One lock per seat	Parallel reservations become possible

This is the central DDD lesson for software architects:

> aggregate boundaries are scaling decisions, not only modeling decisions

Domain events and cross-aggregate rules

When something meaningful happens, record it in the language of the domain.

Examples:

- `SeatReserved`
- `ReservationConfirmed`
- `ReservationReleased`

Why they matter:

- they let other contexts react without direct coupling
- they make eventual consistency explicit
- they prepare the ground for later topics like saga coordination

Some rules do not fit inside one aggregate

RocketTickets constraint:

> A customer may hold at most 2 seats simultaneously.

If each `Seat` is its own aggregate, that rule spans multiple aggregates.

So now you have a distributed-systems problem, not a modeling mistake.

Possible responses:

- application-service check with a race window
- distributed counter or lock
- compensation via process manager / saga

Discovery practices that reveal the model

Event Storming

Use it to discover events, commands, hotspots, and emerging boundaries with domain experts in the room.

BDD / Gherkin

Use it to make the language executable. If the domain expert rejects the scenario text, the model is already drifting.

Example Mapping is still useful, but here the key point is simpler: discovery practices are boundary-discovery tools, not team rituals for their own sake.

Ubiquitous language in executable form

```
Feature: Seat reservation during a flash sale

  Scenario: Reserving an available seat
    Given a concert event with 500 available seats
    And a flash sale has started
    When a customer reserves seat 42
    Then the seat is held for 5 minutes
    And seat 42 is no longer available to other customers

  Scenario: Attempting to reserve an already held seat
    Given seat 42 is held by another customer
    When a second customer attempts to reserve seat 42
    Then the reservation is rejected
```

Every term here should survive the journey from workshop, to code, to test, to architecture discussion.

Debrief – what leaves this room with you

By the end of this lecture, you should have:

- a first subdomain map
- candidate bounded contexts
- at least one explicit context relationship and translation boundary
- first aggregate boundaries
- a list of risky cross-aggregate rules

These will come in handy later when you need to:

- reason about architectural features and quality attributes
- justify why some boundaries deserve stronger protection than others
- choose where consistency, coupling, and scalability risks actually live

What this lets you do next

Output from today	Later architectural use
Subdomain map	Decide where performance, modifiability, or security matter most
Bounded contexts	Ask whether the chosen boundaries reduce coupling or create friction
Context relationships / ACLs	Protect the core model from external APIs and unstable contracts
Aggregate boundaries	Evaluate whether concurrency and consistency costs are acceptable
Cross-aggregate rules	Recognize where eventual consistency or coordination mechanisms will be needed

This is the practical payoff of Domain-Driven Design: it gives architecture something concrete to reason about.

📖 Revisit – Home Reading

Domain-Driven Design

The live lecture now covers strategic design, bounded contexts, context relationships, aggregates, and domain events at a higher level. This section keeps the fuller vocabulary, decision rules, implementation patterns, and workshop detail.

DDD vocabulary — full reference

Term	Definition
Domain	The subject area the software addresses — the business activity it models
Subdomain	A coherent part of the domain with its own logic, data, and team ownership
Core Subdomain	Where the business wins or loses; high complexity, high differentiation; build and protect in-house
Generic Subdomain	A solved problem; buy, use open-source, or outsource; do not invest engineering effort here
Supporting Subdomain	Necessary but not differentiating; simple implementations only; never over-engineer
Ubiquitous Language	The shared, precise vocabulary between domain experts and engineers, used in conversation, docs, and code
Bounded Context	An explicit boundary within which a domain model is defined and consistent
Context Map	A diagram showing the relationships and translation patterns between bounded contexts
Entity	A domain object with a persistent identity that survives state changes
Value Object	An immutable domain object defined entirely by its attributes; no identity
Aggregate	A cluster of domain objects treated as a consistency unit, accessed only through its root
Aggregate Root	The entity that controls all access into the aggregate and enforces invariants
Repository	An abstraction that loads and saves aggregates; hides persistence details from the domain
Domain Event	A record that something meaningful happened in the domain; named in past tense

Context Map – integration patterns

When two bounded contexts need to communicate, they need a translation strategy.

Pattern	When to use
Shared Kernel	Two teams share a small, agreed subset of the model. High coordination cost.
Customer–Supplier	Downstream team (customer) depends on upstream team (supplier). Upstream must not break downstream contracts.
Conformist	Downstream adopts upstream model as-is. No translation. Used when upstream has no incentive to support downstream.
Anti-Corruption Layer (ACL)	Downstream translates upstream model into its own. Protects the bounded context from external model pollution.
Open Host Service	Upstream publishes a well-defined protocol for multiple consumers. API design applies here.
Published Language	A shared formal language (e.g. JSON schema, Protobuf) as integration medium.

The most important pattern for most systems: **Anti-Corruption Layer (ACL)**. Wrap external systems at the boundary. Never let a third-party data structure leak into your domain model.

What the ACL contains

The ACL is the translation layer between two bounded contexts. It receives data from the upstream context in the upstream's format, and converts it into the downstream context's own types. It is the only place in the codebase that knows about the upstream's model.

Aggregate design rules

Getting aggregate boundaries wrong is one of the most common DDD mistakes.

Rule 1: Design for consistency, not convenience An aggregate boundary is a *consistency boundary*. Everything inside one aggregate is updated atomically. If two things must always be consistent, they belong in the same aggregate.

Rule 2: Reference other aggregates by identity only Never hold a reference to another aggregate's object. Hold its ID. This prevents loading unnecessary data and removes implicit coupling between aggregates.

Rule 3: Prefer small aggregates Large aggregates cause contention. If multiple users can modify different parts of an aggregate simultaneously, it should be split.

Rule 4: Enforce invariants at the root The aggregate root is the only place domain rules are enforced. No property setter should be public on an entity inside an aggregate.

Rule 5: One aggregate per transaction Each database transaction should modify at most one aggregate. Cross-aggregate operations require eventual consistency and domain events.

Entity vs Value Object — decision guide

Question	Entity	Value Object
Does it need a unique identifier?	Yes	No
Can two instances with the same data be distinct?	Yes	No — they are equal
Does its identity survive state changes?	Yes	Irrelevant — create a new one
Is it mutable?	Usually yes	Always no
Example	`Customer`, `Order`, `Seat`, `Reservation`	`Money`, `EmailAddress`, `Address`, `DateRange`, `SeatNumber`

When in doubt: favour Value Objects. They are simpler, safer, and easier to test. A common mistake: making `Address` an entity when it should be a value object. A customer can move — that produces a new `Address`, it does not mutate the old one.

Aggregate design – C# patterns

```
// ✅ Aggregate root with encapsulated collection
public class Order {  // aggregate root
    private readonly List<OrderLine> _lines = new();
    public IReadOnlyList<OrderLine> Lines ⇒ _lines.AsReadOnly();
    public Guid Id { get; private set; }
    public Guid CustomerId { get; private set; }   // reference by Id — not a Customer
    public OrderStatus Status { get; private set; }

    public void AddLine(Guid productId, int quantity, Money unitPrice) {
        if (Status ≠ OrderStatus.Draft)
            throw new InvalidOperationException("Cannot modify a submitted order");
        _lines.Add(new OrderLine(productId, quantity, unitPrice));
    }

    public void Submit() {
        if (!_lines.Any()) throw new InvalidOperationException("Order has no lines");
        Status = OrderStatus.Submitted;
        // 🔍 raise a domain event here in a full implementation
    }
}

public class OrderLine {  // entity inside the aggregate — not accessible from outside
    internal OrderLine(Guid productId, int quantity, Money unitPrice) { … }
}
```

Domain Events in C#

Domain events record that something meaningful happened. They are raised inside the aggregate and dispatched after the transaction commits.

```
// ✅ Domain event — named in past tense, carries the relevant facts
public record SeatReserved(
    Guid SeatId,
    Guid CustomerId,
    Guid EventId,
    DateTimeOffset ReservedAt,
    TimeSpan HoldDuration);

// ✅ Aggregate raises events internally
public class Seat {
    private readonly List<IDomainEvent> _events = new();
    public IReadOnlyList<IDomainEvent> Events => _events.AsReadOnly();

    public void Reserve(Guid customerId) {
        if (Status ≠ SeatStatus.Available)
            throw new SeatUnavailableException(Id);
        Status = SeatStatus.Held;
        _events.Add(new SeatReserved(Id, customerId, EventId,
            DateTimeOffset.UtcNow, TimeSpan.FromMinutes(5)));
    }
}
```

Event Storming — the full process

Event Storming is a workshop format for discovering a domain model collaboratively. Invented by Alberto Brandolini.

Who: Domain experts + engineers + product managers. 5–10 people. No laptops. No slides.

What you need: A long paper roll fixed to a wall (the timeline grows left to right). Five colours of sticky notes.

Colour	Represents	Example
🟠 Orange	**Domain Events** — things that happened, past tense	`SeatReserved`, `PaymentFailed`
🔵 Blue	**Commands** — what triggered the event	`Reserve Seat`, `Process Payment`
🟡 Yellow	**Actors** — who issues the command	`Customer`, `Admin`, `System`
🟣 Purple	**Policies** — "whenever X, do Y"	`When PaymentFailed → Release Seat`
🔴 Red	**Hotspots** — disagreements, uncertainties, open questions	

Facilitation sequence

1. Start with Domain Events: ask *"what happened in this process?"* — one event per orange sticky, past tense only
2. For each event: add the blue Command that caused it and the yellow Actor who issued it
3. Add purple Policies where one event automatically triggers another
4. Place red Hotspots wherever people disagree on a term, a sequence, or a rule
5. Step back: clusters of related events reveal aggregate boundaries; places where language shifts reveal bounded context boundaries

Gherkin — syntax and structure

Gherkin is a structured natural language for writing executable specifications. It is the language of Behaviour-Driven Development (BDD). A Gherkin file is both a specification and a test — it can be run directly against the code.

File structure

```
Feature: <name of the capability being specified>
  <optional description — written for a business reader>

  Background:            # optional: steps that run before every scenario
    Given ...

  Scenario: <name of a specific case>
    Given <precondition — the world state before the action>
    When  <action — what the actor does>
    Then  <outcome — what must be true afterwards>
    And   <additional outcome>

  Scenario Outline: <parameterised case>
    Given a seat with status <status>
    When a customer attempts to reserve it
    Then the result is <result>

    Examples:
      | status    | result   |
      | Available | Reserved |
      | Held      | Rejected |
      | Sold      | Rejected |
```

Keywords: `Feature` , `Background` , `Scenario` , `Scenario Outline` , `Given` , `When` , `Then` , `And` , `But` , `Examples`

Gherkin — what makes it good

Good Gherkin reads in the ubiquitous language of the domain. Bad Gherkin reads like a UI script.

❌ UI-focused (bad)	✅ Domain-focused (good)
`When I click the "Reserve" button`	`When a customer reserves seat 42`
`Then I see "Booking confirmed" on screen`	`Then the seat is held for 5 minutes`
`And the green banner appears`	`And the customer receives a payment window notification`

Rules for good Gherkin

- Write in the third person: *a customer*, *an admin*, not *I*
- Name things as the domain names them — `reserve`, not `book` or `select`
- One `When` per scenario — if you need two actions, you have two scenarios
- `Given` sets state, `When` performs exactly one action, `Then` asserts outcome
- Avoid implementation detail: no button names, no URLs, no field IDs

Why it matters for DDD: every term in a Gherkin file is a claim about the ubiquitous language. If a product manager reads a scenario and disagrees with a word, that is a domain modelling conversation — not a QA issue.

Gherkin — RocketTickets extended scenarios

```
Feature: Seat reservation during a flash sale

  Background:
    Given a concert event with 500 available seats
    And a flash sale is in progress

  Scenario: Reserving an available seat
    When a customer reserves seat 42
    Then seat 42 is held for 5 minutes
    And the customer receives a payment window notification
    And seat 42 is no longer available to other customers

  Scenario: Reservation expires without payment
    Given seat 42 is held by a customer
    When the 5-minute hold expires without payment
    Then seat 42 becomes available again
    And the customer's reservation is cancelled

  Scenario: Attempting to reserve a held seat
    Given seat 42 is held by another customer
    When a second customer attempts to reserve seat 42
    Then the reservation is rejected
    And the second customer is offered the next available seat

  Scenario Outline: Hold duration by event tier
    Given a <tier> event
    When a customer reserves a seat
    Then the seat is held for <duration>

    Examples:
```

Gherkin in C# – SpecFlow

SpecFlow binds Gherkin scenarios to C# step definitions. Each step maps to a method.

```
[Binding]
public class SeatReservationSteps {
    private readonly ReservationService _service;
    private ReservationResult _result;

    [Given(@"a concert event with (\d+) available seats")]
    public void GivenEventWithSeats(int count)
        ⇒ _service.SetupEvent(seatCount: count);

    [When(@"a customer reserves seat (\d+)")]
    public void WhenCustomerReservesSeat(int seatNumber)
        ⇒ _result = _service.Reserve(seatNumber, customerId: Guid.NewGuid());

    [Then(@"seat (\d+) is held for (\d+) minutes")]
    public void ThenSeatIsHeld(int seatNumber, int minutes) {
        Assert.Equal(ReservationStatus.Held, _result.Status);
        Assert.Equal(TimeSpan.FromMinutes(minutes), _result.HoldDuration);
    }
}
```

The step definition uses the domain model directly – `ReservationService`, `ReservationStatus`, `HoldDuration`. No UI, no HTTP, no database in the test itself.

Example Mapping and Domain Storytelling

Two lighter-weight alternatives to Event Storming for smaller scopes.

Example Mapping (from BDD)

Takes a single user story and maps it to:

- Concrete examples (become Gherkin scenarios)
- Business rules (become acceptance criteria)
- Open questions (become follow-up conversations)

Run in 25–30 minutes per story. Produces acceptance criteria in domain language before any code is written.

Domain Storytelling

A domain expert narrates a process using pictographic symbols (actors, work items, activities). The listener draws the story. Misunderstandings surface immediately when the picture does not match what the expert intended. Produces a shared visual vocabulary very fast.

Both techniques share the same goal as Event Storming: force domain ambiguity into the open before it becomes a production incident.

Further reading

- **Learning Domain-Driven Design** — Vlad Khononov (O'Reilly) — strategic + tactical, modern examples
- **Domain-Driven Design: Tackling Complexity in the Heart of Software** — Eric Evans — the original; read chapters 1–4 and 6 first
- **Implementing Domain-Driven Design** — Vaughn Vernon — tactical implementation depth; strong on aggregates and bounded contexts
- **Event Storming** — Alberto Brandolini — the original workshop format; available as a free PDF at eventstorming.com
- **BDD in Action** — John Ferguson Smart (O'Reilly) — Gherkin, Example Mapping, and living documentation in practice
- **Domain Storytelling** — Hofer & Schwentner — the pictographic technique; free resources at domainstorytelling.org